Materia: Programación IV

UNIDAD 6 · RELACIONES Y ESTADO

APUNTE TEÓRICO

React Hooks: useState y useEffect

Revisor de la unidad: Prof. Juan Ignacio Sarmiento

Manejo de estado y efectos en componentes funcionales

1. Introducción a los Hooks de React

React es una biblioteca para construir interfaces de usuario basadas en componentes.

En versiones anteriores de React, ciertas funcionalidades como el manejo del estado o el ciclo de vida del componente solo estaban disponibles en **componentes de clase**.

Con la introducción de los **Hooks**, React permitió utilizar estas capacidades directamente dentro de **componentes funcionales**.

Los hooks son funciones especiales que permiten:

- manejar estado
- ejecutar lógica en distintos momentos del ciclo de vida del componente
- reutilizar lógica entre componentes

Algunos de los hooks más utilizados son:

- `useState`
- `useEffect`
- `useContext`
- `useRef`

Este material se enfoca en los dos hooks fundamentales para comenzar a trabajar con React:

- `useState`, que permite manejar estado dentro de un componente.
- `useEffect`, que permite ejecutar efectos secundarios luego del render.

2. El concepto de estado en React

El **estado** representa información que puede cambiar con el tiempo dentro de un componente.

A diferencia de los valores constantes o variables simples, el estado está conectado con el sistema de renderizado de React.

Cuando el estado cambia, React vuelve a renderizar el componente para actualizar la interfaz.

Esto permite crear **interfaces dinámicas y reactivas**.

Algunos ejemplos de estado dentro de una aplicación pueden ser:

- el valor de un contador
- el texto que un usuario escribe en un formulario
- si un menú está abierto o cerrado
- los datos obtenidos desde una API

El estado es una de las piezas centrales del modelo de React, ya que permite que la interfaz responda automáticamente a las interacciones del usuario.

3. Por qué React necesita estado

En JavaScript tradicional, si una variable cambia, la interfaz del usuario no se actualiza automáticamente.

Por ejemplo, si se tiene una variable contador y se modifica su valor, el navegador no sabe que debe actualizar la pantalla.

React resuelve este problema mediante el uso de **estado**.

Cuando el estado cambia, React detecta ese cambio y vuelve a ejecutar el componente para generar nuevamente la interfaz.

Este proceso se conoce como **re-render**.

El re-render permite que la interfaz siempre refleje el estado actual de la aplicación.

4. El hook useState

Para manejar estado dentro de componentes funcionales se utiliza el hook `useState`.

Este hook permite declarar una variable de estado dentro de un componente.

Su sintaxis general es la siguiente:

```
const [state, setState] = useState(valorInicial)
```

`useState` devuelve dos elementos:

1. El valor actual del estado.

2. Una función que permite actualizar ese estado.

El valor que se pasa dentro de `useState()` representa el **valor inicial del estado**.

Por ejemplo, para crear un contador que empiece en cero:

```
const [count, setCount] = useState(0)
```

En este caso:

- `count` representa el valor actual del contador.
- `setCount` es la función que permite modificar ese valor.

5. Qué ocurre cuando cambia el estado

Cuando se llama a la función que actualiza el estado (por ejemplo `setCount`), React realiza varias acciones:

1. Actualiza el valor del estado.
2. Vuelve a ejecutar el componente.
3. Genera nuevamente el JSX.
4. Actualiza la interfaz en el navegador.

Este proceso ocurre automáticamente y es lo que permite que las aplicaciones React sean **reactivas**.

El desarrollador no necesita manipular directamente el DOM.

React se encarga de actualizar la interfaz en función del estado actual.

6. Actualización basada en el estado previo

En algunos casos el nuevo valor del estado depende del valor anterior.

Para estos casos React permite actualizar el estado utilizando una función.

Por ejemplo:

```
setCount(prev => prev + 1)
```

Aquí `prev` representa el valor anterior del estado.

Este patrón es recomendable cuando el nuevo valor depende del anterior, ya que asegura que React utilice el valor más reciente disponible.

7. Estado y componentes

En React el estado **vive dentro de un componente**.

Ese componente es responsable de manejar ese estado y controlar cómo cambia.

Sin embargo, otros componentes pueden necesitar acceder a esa información.

En estos casos se utiliza el sistema de **props**.

Las props permiten pasar información desde un componente padre hacia componentes hijos.

Este modelo se conoce como **flujo de datos unidireccional**.

El estado se mantiene en el componente padre, y los hijos reciben los datos necesarios para renderizar la interfaz o ejecutar acciones.

8. Flujo del estado en una aplicación React

Cuando un usuario interactúa con la aplicación, ocurre el siguiente proceso:

```
Interacción del usuario
   ↓
Evento (click, input, etc.)
   ↓
Actualización del estado
   ↓
React vuelve a renderizar el componente
   ↓
La interfaz se actualiza
```

Este flujo es fundamental para entender cómo funcionan las aplicaciones React.

9. Introducción a useEffect

Además del estado, muchas aplicaciones necesitan ejecutar lógica adicional cuando ocurre algo dentro del componente.

Por ejemplo:

- obtener datos desde una API
- escuchar eventos del navegador
- sincronizar datos con almacenamiento local
- ejecutar temporizadores

Este tipo de operaciones se conocen como **efectos secundarios**.

Para manejar estos efectos React proporciona el hook `useEffect`.

10. Qué es un efecto secundario

Un **efecto secundario** es cualquier operación que ocurre fuera del proceso de renderizado de la interfaz.

En React primero se renderiza la interfaz y luego se ejecutan los efectos.

Esto garantiza que la pantalla se actualice rápidamente antes de ejecutar lógica adicional.

11. Sintaxis de useEffect

La sintaxis general de `useEffect` es la siguiente:

```
useEffect(() => {
  // código del efecto
}, [dependencias])
```

`useEffect` recibe dos argumentos:

1. Una función que contiene el código del efecto.
2. Un array de dependencias que controla cuándo se ejecuta el efecto.

12. El array de dependencias

El comportamiento de `useEffect` depende del array de dependencias.

Existen tres comportamientos principales.

Sin array de dependencias

```
useEffect(() => {
})
```

El efecto se ejecuta después de **cada render**.

Array vacío

```
useEffect(() => {
}, [])
```

El efecto se ejecuta **solo una vez**, cuando el componente se monta.

Dependencias específicas

```
useEffect(() => {
}, [count])
```

El efecto se ejecuta cada vez que cambia el valor de `count`.

13. Flujo de ejecución de useEffect

El orden en que React ejecuta estas operaciones es el siguiente:

```
Evento del usuario
   ↓
Cambio de estado
   ↓
React renderiza el componente
   ↓
React actualiza la interfaz
   ↓
React ejecuta useEffect
```

Este flujo garantiza que los efectos se ejecuten **después de que la interfaz ya está actualizada**.

14. Cleanup function

Un efecto también puede devolver una función de limpieza llamada **cleanup function**.

Su sintaxis es la siguiente:

```
useEffect(() => {
  return () => {
    // limpieza
  }
}, [])
```

Esta función se ejecuta cuando:

- el componente se desmonta
- antes de que el efecto vuelva a ejecutarse

15. Para qué se usa cleanup

La función de limpieza se utiliza para liberar recursos que ya no son necesarios.

Por ejemplo:

- cancelar temporizadores
- remover event listeners
- cancelar peticiones a APIs
- cerrar conexiones abiertas

No todos los efectos requieren una función de limpieza, pero es importante conocer este mecanismo.

16. Conclusión

Los hooks `useState` y `useEffect` son dos de las herramientas más importantes dentro del ecosistema de React.

`useState` permite manejar estado dentro de los componentes, haciendo posible crear interfaces dinámicas y reactivas.

`useEffect` permite ejecutar lógica adicional luego del render del componente y reaccionar a cambios en el estado.

Comprender correctamente estos dos hooks es fundamental para desarrollar aplicaciones modernas con React, ya que forman la base sobre la cual se construyen la mayoría de las funcionalidades de una aplicación.

Bibliografía

1. Banks, A., & Porcello, E. (2020). Learning React (2.ª ed.). O'Reilly Media.
2. Cherny, B. (2019). Programming TypeScript. O'Reilly Media.
3. Microsoft. (s. f.). TypeScript handbook. Recuperado el 2 de octubre de 2026, de https://www.typescriptlang.org/docs/handbook
4. React. (s. f.). React documentation. Recuperado el 2 de octubre de 2026, de https://react.dev